Sharing Repositories and Collaborating: Git Remotes, GitHub, and Shared Drives

Nathan Green (UCL)

Session Overview (30 Minutes)

In our previous sessions, we learned how to work with Git on our own computer: creating a local repository, staging changes, committing snapshots, and inspecting history.

Today we take the big leap: **working with others and sharing code**.

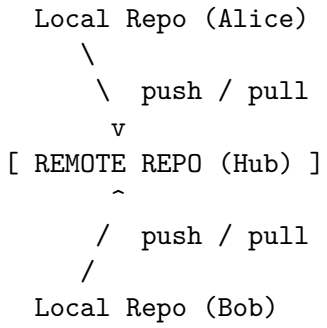
- **Part 1 (10 mins): What is a Remote?**
  - Git is distributed: Cloud (GitHub) vs Shared Drives (Network/Local)
  - What is a "bare" repository and why do we need one?
- **Part 2 (15 mins): The Core Collaboration Workflow**
  - The big three commands: `git clone`, `git push`, `git pull`
  - Hands-on interactive exercise: Simulating two collaborators
- **Part 3 (5 mins): Best Practices & Troubleshooting**
  - When pushes get rejected ("Fetch first!")
  - Golden rules for smooth team collaboration

---

1. Demystifying "Remotes"

What is a Remote Repository?

A **remote** is simply another copy of your Git repository located somewhere else that your local repository can talk to. It acts as the central hub or "single source of truth" for a project.



Instead of emailing files like `analysis_v3_final_NathanEdits.R`, everyone synchronises their local changes with the remote hub.

Git is Distributed: Cloud vs Shared Drive

A common misconception is that **Git requires GitHub**. It does not!

Git is a **distributed** version control system. Your remote repository can live anywhere a file system or network connection exists:

Remote Type	Where it lives	When to use it
GitHub / GitLab / Bitbucket	Cloud servers	Open-source work, public code, non-sensitive projects, teams across different institutions
Shared Network Drive	Internal university/company server (e.g., `S:\ResearchTeam\`)	**Sensitive patient/clinical data**, confidential code, offline environments, strict governance
Local Folder / USB	Another directory on your PC or external drive	Practice, offline backups, personal sync

> **Key takeaway:** The commands (`clone`, `push`, `pull`) are **identical** whether your remote is `https://github.com/...` or `S:/SharedData/MyProject.git`!

What is a "Bare" Repository?

When you create a standard repository with `git init`, you get two things: 1. The hidden `.git` directory (the history and database). 2. The working directory (the visible files you open and edit).

Why central shared remotes must be "bare":

If Alice and Bob push changes to a remote folder that has files checked out, Git would get confused about whose active edits should show on screen.

To solve this, shared central repositories are created with the `--bare` flag:

```
git init --bare
```

- A **bare repository** contains *only* the version control database (no working files to edit directly).
- It serves purely as an exchange post for people to push to and pull from.
- On GitHub, every repository you create is automatically a bare repository behind the scenes!

---

2. The Core Remote Commands

There are three essential commands you will use daily:

1. `git clone <location>`

Downloads an existing remote repository to your local machine, creating a new working folder with the full commit history and setting up the link to the remote automatically.

2. `git push`

Sends your local committed snapshots up to the remote repository so your collaborators can see them.

3. `git pull`

Fetches new commits from the remote repository and integrates them into your current local files.

Checking Your Remotes: `git remote -v`

To see where your current repository is connected, run:

```
git remote -v
```

Output:

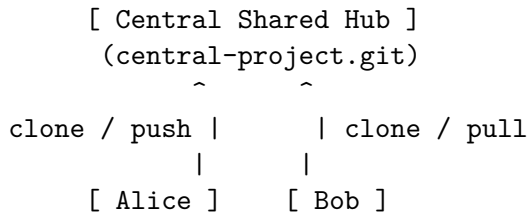
```
origin  https://github.com/ICON-in-R/git-and-github-training.git (fetch)
origin  https://github.com/ICON-in-R/git-and-github-training.git (push)
```

- **`origin`**: This is just the default nickname Git gives to the primary remote repository URL/path you cloned from or connected to.
- It is much easier to type `git push origin main` than typing the whole URL or folder path every time!

---

3. Interactive Hands-On Exercise (15 Mins)

We are going to simulate a full two-person team collaboration right on your computer. No internet or GitHub account required!



Open your terminal (PowerShell, Git Bash, or Terminal).

Step 1: Create the Central Remote Hub (e.g. on a Shared Drive)

First, let's create a shared folder that represents our central server or network drive:

```
# Navigate to your workspace folder
cd ~

# Create a folder for the central shared repository
mkdir central-project.git
cd central-project.git

# Initialise it as a BARE repository
git init --bare
```

(Note: by convention, bare repositories end with `.git` in their folder name).

Step 2: "Alice" Clones and Publishes the First Work

Now pretend to be Alice starting the project on her local machine.

```
# Move back to your home or practice directory
cd ~

# Alice clones the central repo into her own workspace folder
git clone central-project.git alice-workspace

# Alice enters her workspace
cd alice-workspace

# Check the remote connection
git remote -v
```

Now Alice creates the first file and pushes it:

```
# Create a project file
echo "# Collaborative Study" > study-notes.txt

# Stage and commit
git add study-notes.txt
```

```
git commit -m "Initial commit of study notes by Alice"

# Push the main branch to the central repo
git push -u origin main
```

> **Note:** The `-u` flag (upstream) only needs to be used the very first time you push a branch. Afterwards, you can simply type `git push`.

Step 3: "Bob" Joins the Project

Now pretend to be Bob, joining the team from another desk.

```
# Move back to your home or practice directory
cd ~

# Bob clones the central repository into his own workspace
git clone central-project.git bob-workspace

# Bob enters his workspace
cd bob-workspace

# Bob inspects the directory - Alice's file is already here!
ls
cat study-notes.txt
```

Bob now contributes his own work:

```
# Bob adds his thoughts to the file
echo "Bob added: Patient cohort analysis plan" >> study-notes.txt

# Bob stages, commits, and pushes
git add study-notes.txt
git commit -m "Add cohort analysis plan by Bob"
git push
```

Bob's changes are now securely stored in the central hub!

Step 4: Alice Gets Bob's Latest Work

Alice comes back from lunch and wants to see what has been updated.

```
# Switch back to Alice's folder
cd ~/alice-workspace

# Alice pulls the updates from the central hub
git pull

# Check the file and the commit history
cat study-notes.txt
git log --oneline
```

Congratulations! You have just completed a complete distributed Git collaboration cycle: `clone` → `commit` → `push` → `pull`!

---

4. What about GitHub?

If your workplace allows cloud hosting (no restricted/patient-identifiable data), GitHub works the **exact same way**:

Scenario A: Starting from an existing GitHub repository

If a colleague already set up a GitHub repository:

```
git clone https://github.com/organization/project-name.git
cd project-name
# work, commit, then:
git push
git pull
```

Scenario B: Connecting an existing local project to GitHub

If you already started a project locally: 1. Create a new empty repository on github.com (leave "Add README" unchecked). 2. Link your local repo to GitHub: `bash git remote add origin https://github.com/your-username/your-repo.git git branch -M main git push -u origin main`

---

5. Collaboration Golden Rules & Gotchas

Rule #1: "Pull before you push"

Before you begin work for the day, and right before you push your commits, always run:

```
git pull
```

This ensures you have the latest updates and prevents surprises.

Gotcha: The "Rejected Push"

What happens if Alice and Bob both make commits at the same time, and Bob pushes first?

When Alice tries to push:

```
To central-project.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs...
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing...
```

Don't panic! Git is protecting Alice from accidentally overwriting Bob's work.

The Fix: 1. Run `git pull` to bring Bob's commits into Alice's repo. 2. Git will combine them automatically (or ask to resolve any conflicts). 3. Alice can now run `git push` successfully!

Sensitive Data & Governance Checklist

Before pushing code anywhere:

1. **Never commit sensitive data:**
   - Patient health identifiers, survey respondent info, passwords, private API tokens.
2. **Use `.gitignore`:**
   - List data files (`*.csv`, `*.rds`, `*.xlsx`, `data/`) so Git ignores them automatically.
3. **Choose the right remote:**
   - Public code → GitHub.
   - Restricted/Governance-controlled data & code → Internal Shared Drive / Trusted Research Environment (TRE).

Summary: Command Cheat Sheet

Task	Command
Copy remote to local	`git clone <path-or-url>`
Check connected remotes	`git remote -v`
Link local repo to a remote	`git remote add origin <path-or-url>`
Upload local commits	`git push` (or `git push -u origin main` first time)
Download & merge remote commits	`git pull`
Download commits without merging	`git fetch`
Create a central server repo	`git init --bare`

Questions & Discussion

- Have you used shared network drives for code before?
- What data governance constraints do you have in your team?
- Next session preview: **Branching, Pull Requests, and Handling Conflicts!**